

Gen AI Citation Discovery

Technical Report

Group 40

Panadero Palenzuela Raquel (12231231)

Schicho Johann Sebastian (12405408)

Sinka Agnes (12412694)

Beliaeva Tatiana (12432964)

Mitrov Kristijan (52319916)

Abstract

Exploring academic literature is often challenging for students due to the large number of papers, complex terminology, and difficulty in identifying relevant work and open research questions. This project presents *Gen AI Citation Discovery*, a generative AI-based research assistant that supports early-stage literature exploration. The system allows users to upload academic PDF documents, discover papers relevant to a given topic, ask questions grounded in document content, and analyze research gaps and claims across multiple papers.

Description

Ideal User Journey

1. Document Upload

The user uploads one or more academic PDF papers through the document explorer.

2. Indexing and Embedding

Documents are automatically chunked and embedded, and the resulting vectors are stored in a FAISS vector database for semantic search.

3. Topic Discovery

The user asks a question such as “Which papers are relevant to topic X?”. The system returns a ranked list of relevant papers with relevance scores.

4. Paper Inspection

The user opens a selected paper in the integrated PDF viewer and navigates through its content.

5. Deep Analysis

The user asks questions about document content, compares papers, verifies claims, or requests research gaps and future work suggestions.

Technical Approach and Design Choices

The application follows a modular frontend–backend architecture.

Frontend (React + Vite) The frontend provides a three-panel layout consisting of a document explorer, a PDF viewer, and a chat interface. Streaming responses are used to allow interactive communication with the AI agent.

Backend (FastAPI + LangGraph) The backend is implemented using FastAPI and includes a LangGraph-based agent. The agent routes user queries to different tools based on intent, such as document search, comparison, or claim verification. A simplified architecture is shown in Figure 1.

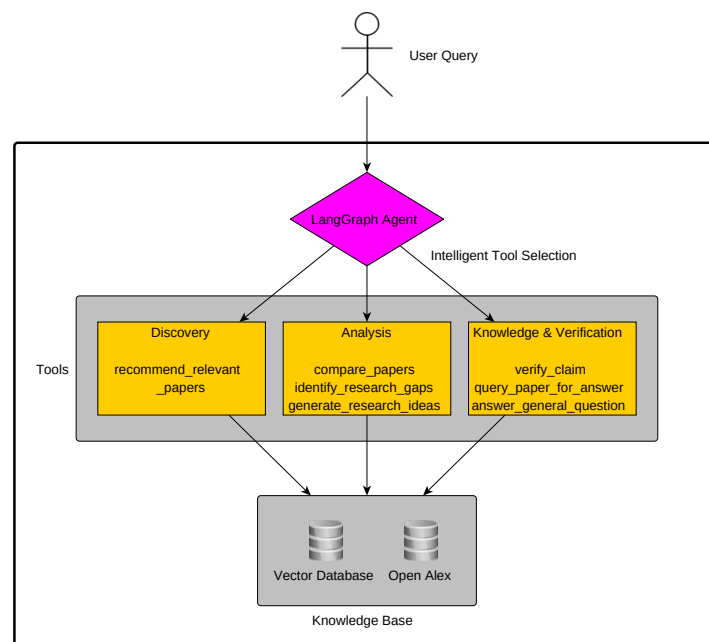


Figure 1: Schematic backend architecture

Retrieval-Augmented Generation (RAG) Uploaded PDFs are split into text chunks and embedded into a FAISS vector database. Relevant chunks are retrieved using semantic similarity and passed to the language model, ensuring that generated answers are grounded in document content.

Implemented RAG Tools The following tools are implemented and integrated into the agent workflow (as listed in Table 1):

External Knowledge Sources The OpenAlex API is integrated to retrieve paper meta-data, related works, and citation information, extending the system beyond locally uploaded documents.

Table 1: Overview of implemented RAG tools and their functionality

Type	Name	Functionality
Discovery	<code>recommend_relevant_papers</code>	Suggests papers relevant to a given topic.
Analysis	<code>compare_papers</code>	Performs comparative analysis of papers based on specific aspects.
	<code>identify_research_gaps</code>	Analyzes multiple papers to identify common research gaps.
	<code>generate_research_ideas</code>	Identifies limitations and suggests directions for future work.
Knowledge	<code>verify_claim</code>	Checks whether a claim is supported, contradicted, or not addressed by the indexed papers.
	<code>query_paper_for_answer</code>	Queries a specific paper using its file path.
	<code>answer_general_question</code>	Searches across all indexed documents.

Challenges and Technical Obstacles

- LangChain advertises interoperability across LLM services, integrating multiple services in practice was still challenging.
- We spent a lot of time configuring and prompting the agent, so that it would consistently choose the appropriate tools based on the user’s intent and query context proved difficult.
- Extending the toolset destabilized the agent’s behavior (missing responses or inconsistent outputs), which made debugging and interpreting routing decisions harder.
- Implementing streaming responses was more complex than expected, because we had to reliably separate streamed text from accompanying metadata.
- We experimented with switching from Gemini to TU Wien Mistral. However, replacing the embedding model led to noticeable quality differences, so we decided to keep Gemini embeddings. Both LLMs are kept to support the chat functionality.
- The system still suffers from high latency (especially when using the TU LLM), and performance optimization remains an open issue.

Evaluation

The system was evaluated by manually testing each tool on several example use cases and scoring the results using four metrics: relevance, grounding, correctness, and latency. Scores were normalized between 0 and 1 and averaged per tool. Overall, the system produced relevant and correct outputs that were generally well grounded in the provided documents. In some cases, responses showed high latency, and citations or references were only partially correct, especially for more complex queries.

The final overall system score of 0.83 indicates that the system works well, with room for further performance and reliability improvements. The averaged scores per tool are summarized in Table 2.

Table 2: Average evaluation scores per tool based on manual testing

Tool	N	Relevance	Grounding	Correctness	Latency	Overall
recommend_relevant_pprs	2	1.00	1.00	0.95	0.60	0.89
answer_general_question	2	0.90	0.75	1.00	0.85	0.88
query_paper_for_answer	1	1.00	1.00	1.00	0.50	0.88
compare_papers	1	1.00	1.00	1.00	0.30	0.83
identify_research_gaps	1	1.00	0.80	0.70	0.30	0.70
generate_research_ideas	1	0.80	0.50	0.80	0.50	0.65
verify_claim	2	1.00	1.00	1.00	0.50	0.88

Results

The system successfully identified relevant papers and generated document-grounded answers. Research gap detection, paper comparison, and claim verification produced meaningful and coherent results, although outputs were sometimes high-level and were taking longer than expected.

Reflection

Human Agency

The system increases human agency by supporting users in exploring and analyzing academic literature more efficiently, while leaving interpretation and critical judgment to the user. The tool does not take away any decision making from the person using it.

Future Development

The system could be developed further. Future work mainly would focus on improving the user interface, adding richer visualizations, and enhancing the presentation of analysis results rather than expanding core AI functionality.